

Java to **GO**

how to make programming **fun*** again?



Michał Dubel

*** fast, universal, neat concurrency model... etc.**



TRANSITION
TECHNOLOGIES

About me



Michał Dubel

Software Architect @ Transition Technologies PSC

dubel.dev

First of all – say NO to hype-driven programming!



Secondly – why new language?



<https://learnyouahaskell.com/>

My job at CISCO



ACCEDIAN



_experience digital transition!



TRANSITION
TECHNOLOGIES

Foundations....

2007 started at Google

2009 announced publicly



Robert Griesemer, Rob Pike, and **Ken Thompson**



2012 open source 1.0 released

Key features (similarities with Java)

- C-based keywords
- static typing
- type inference
- Compiled
- GC
- memory safety

Key features (differences from Java)

- no VM
- structs, no objects
- No inheritance*
- dependencies management
- Pointers
- Concurrency
- no exceptions

Go reserved keywords – only 25 of them

- break
- default
- func
- Interface
- select
- case
- else
- import
- return
- var
- defer
- go
- map
- struct
- chan
- goto*
- for
- package
- switch
- const
- fallthrough
- if
- range
- type
- continue

Go – Use cases (aka right tool for a right job)



Go – Use cases (aka right tool for a right job)

- Cloud & Network Services



Go – Use cases (aka right tool for a right job)

- Cloud & Network Services
- Command-line Interfaces (CLIs)



Go – Use cases (aka right tool for a right job)

- Cloud & Network Services
- Command-line Interfaces (CLIs)
- Web Development



Go – Use cases (aka right tool for a right job)

- Cloud & Network Services
- Command-line Interfaces (CLIs)
- Web Development
- Development Operations & Site Reliability Engineering



What can be built?



Prometheus



Grafana



ngrok



Terraform

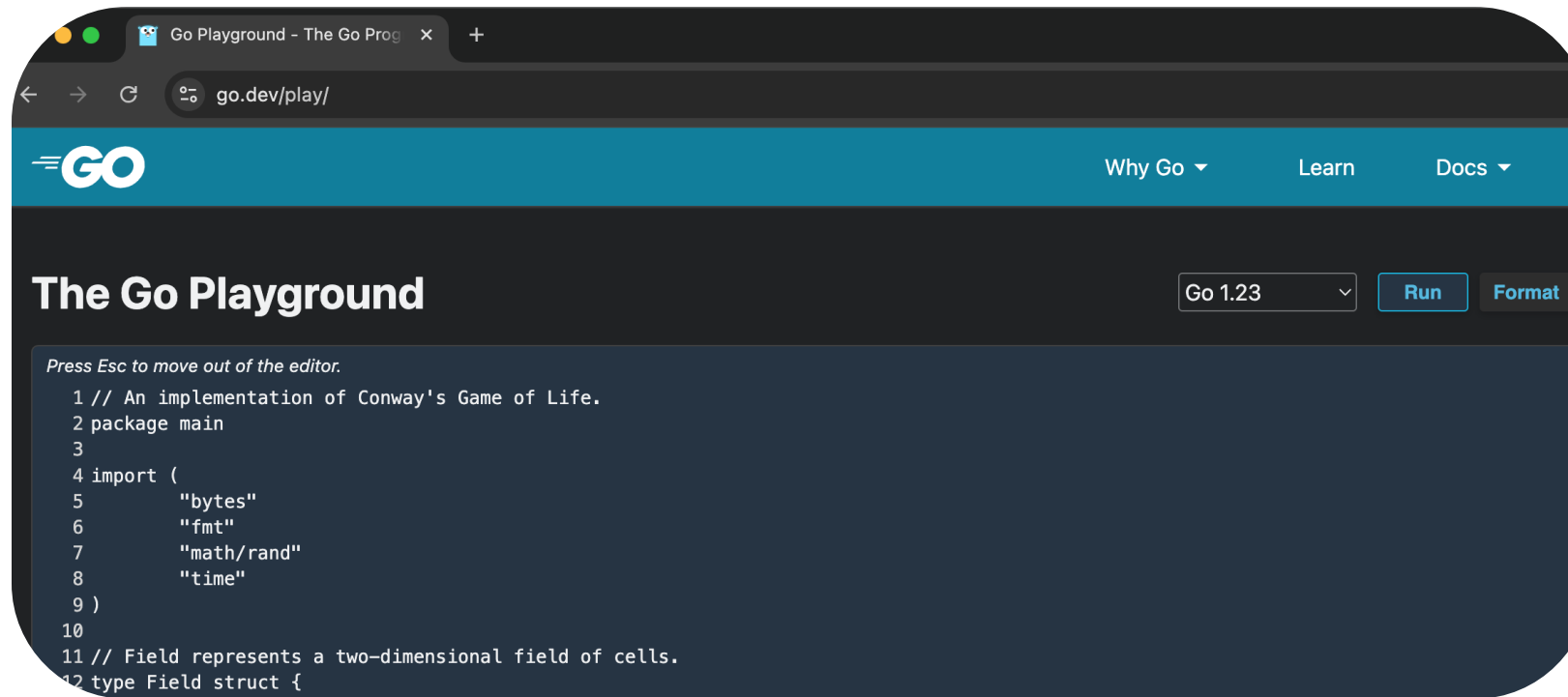


Dgraph

Use Go(lang) to improve search results

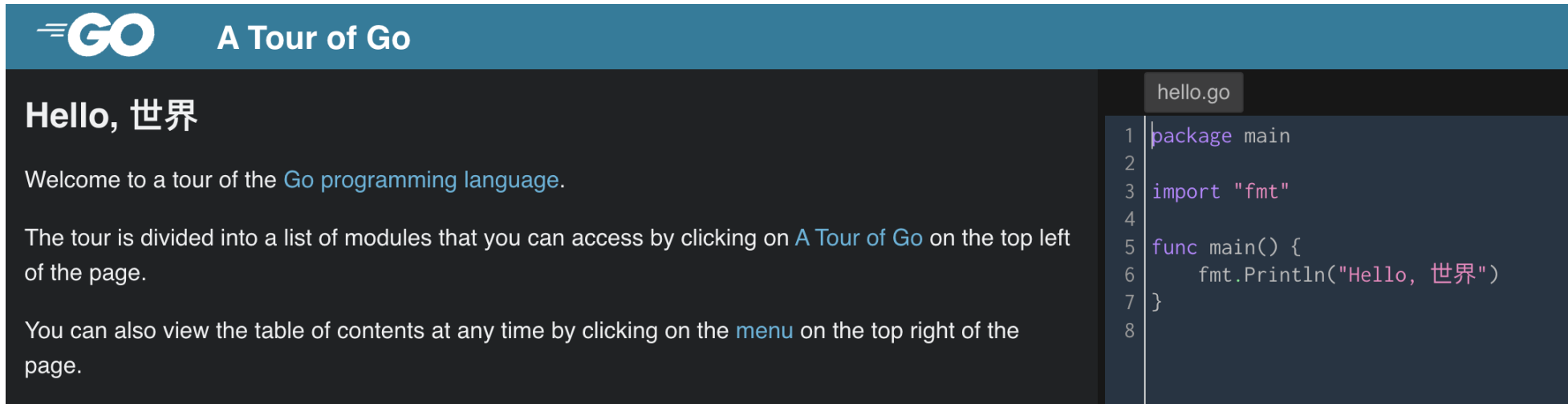


How to start – fast track



<https://go.dev/play>

Combine it with Tour of Go



The screenshot shows the 'A Tour of Go' website. The header has the Go logo and the title 'A Tour of Go'. The main content area is titled 'Hello, 世界' and contains the following text:

Welcome to a tour of the [Go programming language](#).

The tour is divided into a list of modules that you can access by clicking on [A Tour of Go](#) on the top left of the page.

You can also view the table of contents at any time by clicking on the [menu](#) on the top right of the page.

On the right side, there is a code editor window titled 'hello.go' containing the following Go code:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Hello, 世界")
7 }
8
```

<https://go.dev/tour>

How to start? For real this time

```
mike@pierwszymbp2015:~/tmp
> go mod init github.com/dubel/java-to-go-devconf
go: creating new go.mod: module github.com/dubel/java-to-go-devconf
> go build ./cmd/jtogo
> go run ./cmd/jtogo
Hello Gophers! It's DevConf time!
```

```
go.mod
module github.com/dubel/java-to-go-devconf

go 1.23.2
```

```
main.go
package main

import "fmt"

func main() {

    fmt.Println("Hello Gophers! It's DevConf time!")
}
```

codetoimg.com

codetoimg.com

Shipping the binary



GraalVM™  ?

Multiple return values

multiple.go

```
// sumAndDiff takes two integers and returns their sum and difference.
func sumAndDiff(a, b int) (int, int) {
    sum := a + b
    diff := a - b
    return sum, diff
}

func main() {
    x, y := 10, 5
    sum, diff := sumAndDiff(x, y)
    fmt.Printf("Sum: %d, Difference: %d\n", sum, diff)

    // Example of ignoring the difference
    sumOnly, _ := sumAndDiff(x, y)
    fmt.Printf("Sum only: %d\n", sumOnly)
}
```

codetoimg.com

Supported platforms – cross compilation

```
mike@pierwszymbp2015:~/tmp
> go tool dist list | column
```

aix/ppc64	dragonfly/amd64	ios/amd64	linux/loong64	linux/riscv64	openbsd/amd64	plan9/arm
android/386	freebsd/386	ios/arm64	linux/mips	linux/s390x	openbsd/arm	solaris/amd64
android/amd64	freebsd/amd64	js/wasm	linux/mips64	netbsd/386	openbsd/arm64	wasip1/wasm
android/arm	freebsd/arm	linux/386	linux/mips64le	netbsd/amd64	openbsd/ppc64	windows/386
android/arm64	freebsd/arm64	linux/amd64	linux/mipsle	netbsd/arm	openbsd/riscv64	windows/amd64
darwin/amd64	freebsd/riscv64	linux/arm	linux/ppc64	netbsd/arm64	plan9/386	windows/arm
darwin/arm64	illumos/amd64	linux/arm64	linux/ppc64le	openbsd/386	plan9/amd64	windows/arm64

Apple ~/tmp

```
mike@pierwszymbp2015:~/tmp/cmd/jtogo
> GOOS=windows GOARCH=amd64 go build
> ls -la
total 4360
drwxr-xr-x  4 mike  staff   128 Oct 10 18:31 .
drwxr-xr-x  3 mike  staff    96 Oct  6 19:23 ..
-rwxr-xr-x  1 mike  staff 2227200 Oct 10 18:31 jtogo.exe
-rw-r--r--  1 mike  staff    96 Oct  6 19:30 main.go
```

Apple ~/tmp/cmd/jtogo



Let's start.... with a rant

```
gorilla.go

func (w *messageWriter) ncopy(max int) (int, error) {
    n := len(w.c.writeBuf) - w.pos
    if n <= 0 {
        if err := w.flushFrame(false, nil); err != nil {
            return 0, err
        }
        n = len(w.c.writeBuf) - w.pos
    }
    if n > max {
        n = max
    }
    return n, nil
}
```

```
nn.go

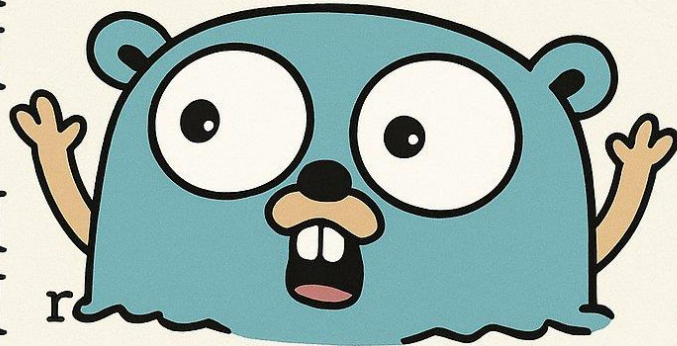
nn := len(p)
for len(p) > 0 {
    n, err := w.ncopy(len(p))
    if err != nil {
        return 0, err
    }
    copy(w.c.writeBuf[w.pos:], p[:n])
    w.pos += n
    p = p[n:]
}
return nn, nil
```



codetoimg.com

Let's start.... with a rant – cd.

```
| if err != nil { return err } if err != nil { ret  
return err { return err } if err != nil { return  
if err != nil { return err } if err != nil { return e  
if err != nil { return err } if err != nil { return  
if err != nil { return err } return err } return er  
if err != nil { return err      return err} return er  
if err != nil {                      return err  
if err != nil { return err } return err }  
if err != nil { { return err  
if err != nil { r return err |  
if err != nil { return err } return err return err  
if err != nil { return err } return err } return er
```



Idioms! Idioms – everywhere!

O'REILLY®

Learning Go

An Idiomatic Approach to
Real-World Go Programming

Second
Edition



Jon Bodner



GO commaok.go

```
myMap := map[string]int{
    "apple": 2,
    "banana": 4,
    "orange": 6,
}

if value, ok := myMap["banana"]; ok {
    fmt.Printf("Key 'banana' is present")
} else {
    fmt.Println("Key 'banana' is not present")
}
```

codetoimg.com

Basic syntax and concepts



Type inference



GO Inference.go

```
func main() {  
  
    var x int = 41  
  
    // could become  
  
    var x = 41  
  
    // but most probable it will become this  
  
    x := 41  
  
}
```

codetoimg.com

Receiver functions – or how to create a method

```
receivers.go

type Vertex struct {
    X, Y float64
}

func (v Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}

func (v *Vertex) Scale(f float64) {
    v.X = v.X * f
    v.Y = v.Y * f
}

func main() {
    v := Vertex{3, 4}
    v.Scale(10)
    fmt.Println(v.Abs())
}
```

codetoimg.com

Implement interfaces... kind of... in a ~~weird~~ different way

```
GO interfaces.go

type I interface {
    M()
}

type T struct {
    S string
}

func (t T) M() {
    fmt.Println(t.S)
}

func main() {
    var i I = T{"hello DevConf"}
    i.M()
}
```

codetoimg.com

Implement interfaces... defined by a consumer

```
client.go

type LogicProvider struct{}

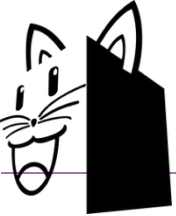
func (lp LogicProvider) Process(data string) string {
    return "" // business logic
}

type Logic interface {
    Process(data string) string
}

type Client struct {
    L Logic
}

func (c Client) Program() {
    // get data from somewhere c.L.Process(data)
}

func main() {
    c := Client{
        L: LogicProvider{},
    }
    c.Program()
}
```



HAI, CAN HAS OOP?

- Encapsulation
- Polymorphism
- Composition over* inheritance



oop.go

```
type Employee struct {  
    Name string  
    ID    string  
    addr string  
}  
  
func (e Employee) Description() string {  
    return fmt.Sprintf("%s (%s)", e.Name, e.ID)  
}  
  
type Manager struct {  
    Employee  
    Reports []Employee  
}
```

codetoimg.com

No inheritance? Composition to the rescue!

GO animal.go

```
type Animal struct {  
    name string  
}  
  
func (a Animal) Speak() {  
    fmt.Println("Hello, I'm", a.name)  
}  
  
type Pet struct {  
    animal *Animal  
    owner  string  
}  
  
func (p *Pet) Speak() {  
    p.animal.Speak()  
    fmt.Println("My owner is", p.owner)  
}
```

GO composition.go

```
func main() {  
    a := &Animal{name: "Fluffy"}  
    p := &Pet{animal: a, owner: "John"}  
  
    p.Speak()  
}
```

Embedding is NOT Inheritance

codetoimg.com

codetoimg.com

Embedding is NOT Inheritance



GO embedFail.go

```
func main() {  
    a := Animal{name: "Fluffy"}  
    pet := Pet{animal: a, owner: "John"}  
    pet.Speak()  
  
    // can't do  
    var animalFail Animal = pet  
    //can be done  
    var animalOk Animal = pet.animal  
  
}
```

codetoing.com

Embedding is NOT Inheritance

```
inner.go

type Inner struct {
    A int
}

func (i Inner) IntPrinter(val int) string {
    return fmt.Sprintf("Inner: %d", val)
}

func (i Inner) Double() string {
    return i.IntPrinter(i.A * 2)
}

type Outer struct {
    Inner
    S string
}

func (o Outer) IntPrinter(val int) string {
    return fmt.Sprintf("Outer: %d", val)
}
```

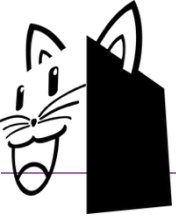
codetoimg.com

```
inner.go

func main() {
    o := Outer{
        Inner: Inner{
            A: 10},
        S: "Hello"}
    fmt.Println(o.Double())

    // prints:
    // Inner: 20
}
```

codetoimg.com



HAI, CAN HAS FP?

- First class function support, meaning Function can be:
 - Assigned to variables
 - Passed to arguments
 - Returned from other functions
- Anonymous functions holding surroundings with closures



functional.go

```
type operation func(int, int) int

func apply(op operation, a, b int) int {
    return op(a, b)
}

func add(a, b int) int {
    return a + b
}

func main() {
    addFunc := add
    fmt.Println(apply(addFunc, 1, 2))

    fmt.Println(apply(func(x int, y int) int {
        return x + y
    }, 3, 55))
}
```

Promise of backward compatibility

“Although we expect that the vast majority of programs will maintain this compatibility over time, it is impossible to guarantee that no future change will break any program. This document is an attempt to set expectations for the compatibility of Go 1 software in the future”

Promise of backward compatibility – “for” change Go 1.22

loop.go

```
func main() {
    done := make(chan bool)

    values := []string{"a", "b", "c"}
    for _, v := range values {
        go func() {
            fmt.Println(v)
            done <- true
        }()
    }

    // wait for all goroutines to complete before exiting
    for _ = range values {
        <-done
    }
}
```

go.mod

```
module github.com/dubel/java-to-go-devconf

go 1.21
```

codetoimg.com

- C
- C
- C

codetoimg.com

Promise of backward compatibility – “for” change Go 1.22

loop.go

```
func main() {
    done := make(chan bool)

    values := []string{"a", "b", "c"}
    for _, v := range values {
        go func() {
            fmt.Println(v)
            done <- true
        }()
    }

    // wait for all goroutines to complete before exiting
    for _ = range values {
        <-done
    }
}
```

go.mod

```
module github.com/dubel/java-to-go-devconf

go 1.23.2
```

codetoimg.com

- c
- b
- a

codetoimg.com

Promise of backward compatibility – for change Go 1.22

Closed Bug 1619047 Opened 5 years ago Closed 4 years ago

Let's Encrypt: CAA Rechecking bug

▼ Categories

Product: CA Program ▼

Component: CA Certificate Compliance ▼

Type: task

Priority: Not set Severity: normal

▼ Tracking

Status: RESOLVED FIXED

► People (Reporter: jsha, Assigned: jsha)

► References

► Details (Whiteboard: [ca-compliance] [dv-misissuance])

▼ Attachments

[current-caa-forbids-issuance.txt](#) Details

5 years ago Jacob Hoffman-Andrews

60.96 KB, text/plain

Bottom ↓

Tags ▼

Timeline ▼

https://bugzilla.mozilla.org/show_bug.cgi?id=1619047

Speeding up since Go 1.18

- 1.18 (March 2022):
 - Generics
 - Fuzz Testing
 - go work to easily manage multiple modules
- 1.19 (August 2022)
 - Memory Model updates
 - defer optimizations
 - Performance optimizations
- 1.20 (February 2023)
 - Toolchain and Build Improvements
 - Slices and workspace enhancements
- 1.21 (August 2023)
 - Toolchain, runtime, and libraries
- 1.22 (February 2024)
 - for loop*
 - Runtime enhancements
- 1.23 (August 2024)
 - Iterator for-range
 - Telemetry
 - Iterators
- 1.24 (August 2024)
 - Map swiss table
 - go tool command
- 1.25 (August 2025)
 - GOMAXPROCS – „container-aware”
 - New GC
 - Compiler enhancements



No exceptions – use the if != nil Luke!



```
func main() {  
    file, err := os.Open("devconf-participants.json")  
    if err != nil {  
        log.Fatal(err)  
    }  
    // handle the opened File and do stuff below  
}
```

codetoimg.com

Pointers!



pointers.go

```
func main() {  
    i, j := 42, 1908  
  
    p := &i           // point to i  
    fmt.Println(*p)    // read i through the pointer  
    *p = 55            // set it through the pointer  
    fmt.Println(*p)    // review new value of an i  
    fmt.Println(p)     // what if we print it without *?  
  
    p = &j             // point i to j  
    *p = *p / 2        // make calculations using pointer  
    fmt.Println(j)  
}
```

codetomg.com

Pointer arithmetic – just don't!



unsafe.go

```
func main() {  
    x := [4]int{1, 2, 3, 4}  
    ptr := unsafe.Pointer(&x[0])  
    next := unsafe.Pointer(uintptr(ptr) + unsafe.Sizeof(x[0]))  
    fmt.Println(*(*int)(next))  
  
    // same with Go 1.17 unsafe.Add  
  
    ptr = unsafe.Pointer(&x[0])  
    next = unsafe.Add(ptr, unsafe.Sizeof(x[0]))  
    fmt.Println(*(*int)(next))  
}
```

codetoimg.com

Iota vs enums

iota.go

```
type Direction int

const (
    North Direction = iota
    East
    South
    West
)

func (d Direction) String() string {
    return [...]string{"North", "East", "South", "West"}[d]
}
```

codetoimg.com

iota.go

```
func main() {
    var d Direction = North
    fmt.Print(d)
    switch d {
    case North:
        fmt.Println(" goes up.")
    case South:
        fmt.Println(" goes down.")
    default:
        fmt.Println(" stays put.")
    }
    // Output: North goes up.
}
```

codetoimg.com

Iota vs enums

● ● ●  iotaweird.go

```
const (  
    Field1 = 0  
    Field2 = 1 + iota  
    Field3 = 20  
    Field4  
    Field5 = iota  
)  
func main() {  
    fmt.Println(Field1, Field2, Field3, Field4, Field5)  
}
```

codetoimg.com

Access modifiers

```
main.go

package main

import (
    "fmt"
    mypackage "github.com/dubel/java-to-go-devconf/cmd/jtogo/package"
)

func main() {

    fmt.Println("Hello Gophers! It's DevConf time!")

    fmt.Println(mypackage.ExportedVariable)
    //Below will not work!
    //fmt.Println(mypackage.unExportedVariable)
}
```

```
mypackage.go

package mypackage

var ExportedVariable = "Hello I'm exported!"
var unExportedVariable = "I'm not accessible outside!"
```


Slices



slices.go

```
primes := [6]int{2, 3, 5, 7, 11, 13}

var slice []int = primes[1:4]
fmt.Println(slice)
```

codetoimg.com

More on Slices!



creating.go

```
// Create a slice using make() - length 5, capacity 10
sliceMake := make([]int, 5, 10)
fmt.Println("Slice created using make:", sliceMake)
fmt.Println("Length:", len(sliceMake), "Capacity:", cap(sliceMake))

// Create a slice using a slice literal
sliceLiteral := []int{1, 2, 3, 4, 5}
fmt.Println("Slice from slice literal:", sliceLiteral)

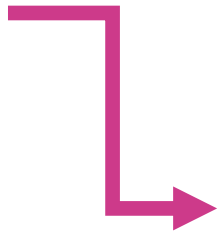
// Create a nil slice
var nilSlice []int
fmt.Println("Nil slice:", nilSlice)
fmt.Println("Is the slice nil?", nilSlice == nil)

// Create a slice from another slice
sliceFromSlice := sliceLiteral[2:4] // Slice of sliceLiteral
fmt.Println("Slice from another slice:", sliceFromSlice)
```

codetoimg.com

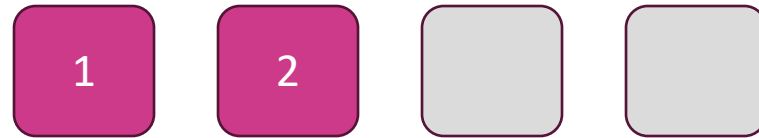
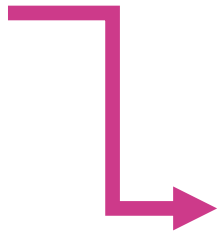
More on Slices!

sliceB → sliceA

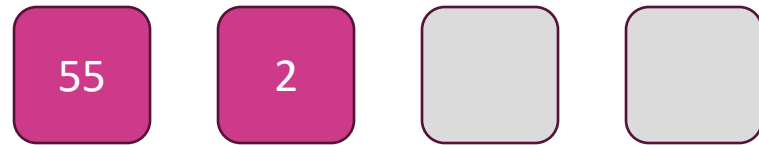


More on Slices!

sliceB → sliceA

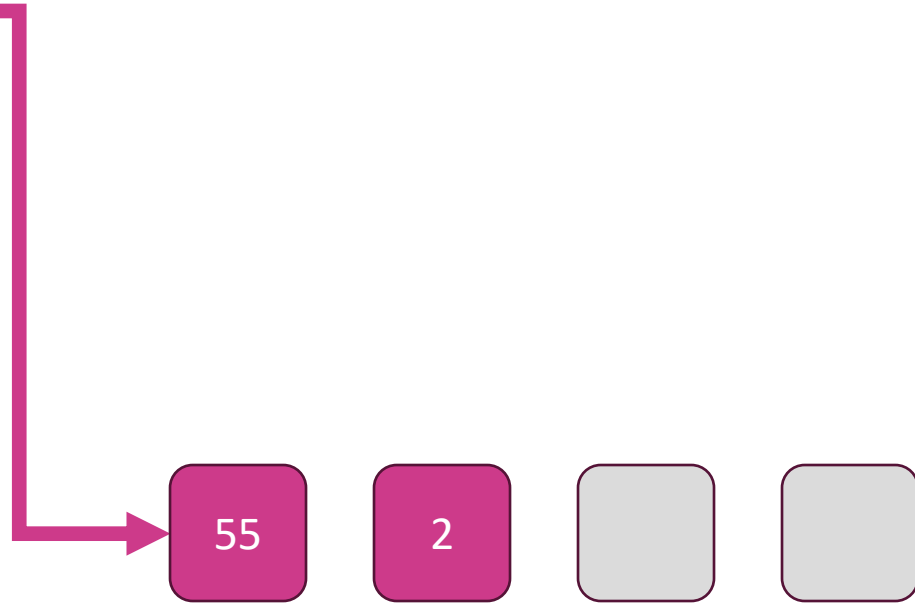


↓ sliceA[0] = 55



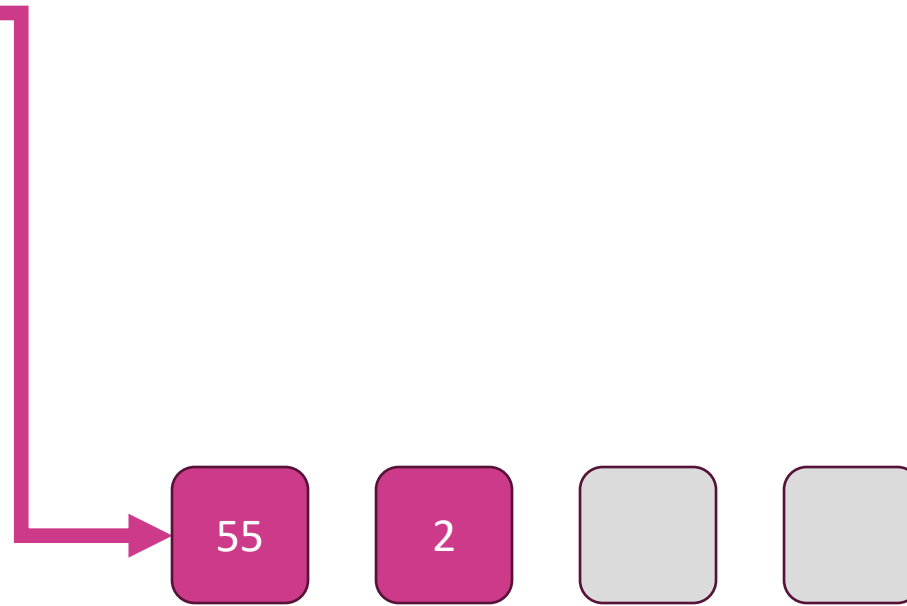
More on Slices!

sliceB → sliceA



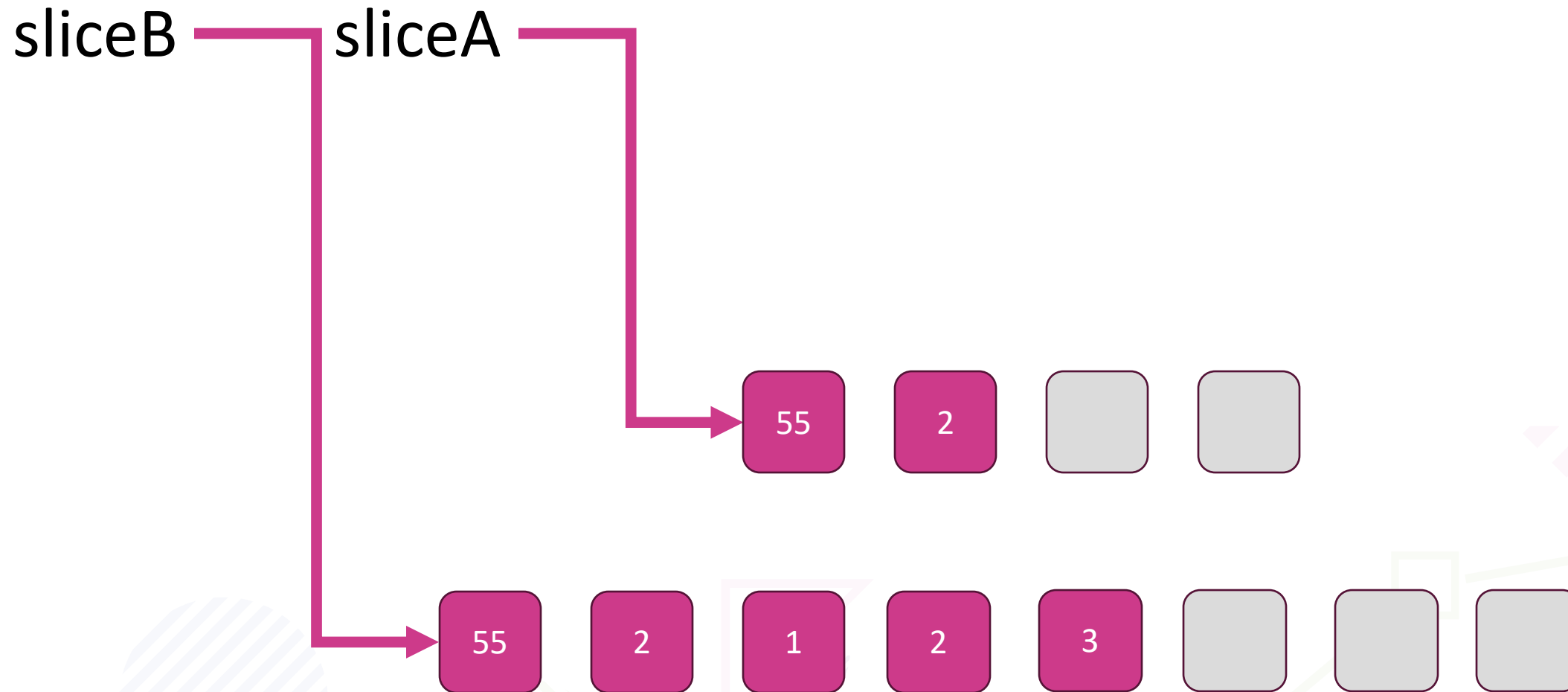
More on Slices!

sliceB → sliceA



↓ sliceB = `append(sliceB, 1, 2, 3)`

More on Slices!



Overloading methods



calculator.go

```
func (c Calculator) Add(a, b int) int {  
    return a + b  
}
```

```
// Attempting to overload Add method (w  
//func (c Calculator) Add(a, b float64)  
//    return a + b  
//}
```



overload.go

```
type SomeStruct struct {  
}
```

```
func (s *SomeStruct) do() {  
    println("do nothing")  
}
```

```
func (s *SomeStruct) doThing(thing string) {  
    println("do thing:", thing)  
}
```

```
func main() {  
    s := &SomeStruct{}  
    s.do()  
    s.doThing("do")  
}
```

Range operator



range.go

```
// Example with an array
numbers := [4]int{1, 2, 3, 4}
for _, number := range numbers {
    fmt.Println(number)
}

// Example with Slice
fruits := []string{"apple", "orange", "cherry"}
for index, fruit := range fruits {
    fmt.Printf("index: %d, fruit: %s\n", index, fruit)
}
```

codetoimg.com

Goroutines



```
func say(s string) {  
    for i := 0; i < 5; i++ {  
        time.Sleep(100 * time.Millisecond)  
        fmt.Println(s)  
    }  
}  
  
func main() {  
    go say("world")  
    say("hello")  
}
```

codeimg.com

Channels

```
func sum(s []int, c chan int) {
    sum := 0
    for _, v := range s {
        sum += v
    }
    c <- sum // send sum to channel c
}

func main() {
    s := []int{7, 2, 8, -9, 4, 0}

    c := make(chan int)
    go sum(s[:len(s)/2], c)
    go sum(s[len(s)/2:], c)
    x, y := <-c, <-c // receive from channel

    fmt.Println(x, y, x+y)
}
```

```
> go run ./channels.go
-5 17 12
```

Buffered and un-buffered channels



buffered.go

```
func processChannel(ch chan int) []int {  
    const conc = 10  
    results := make(chan int, conc)  
    for i := 0; i < conc; i++ {  
        go func() {  
            v := <-ch  
            results <- process(v)  
        }()  
    }  
    var out []int  
    for i := 0; i < conc; i++ {  
        out = append(out, <-results)  
    }  
    return out  
}
```

Syncing using Wait Groups

```
wait.go

func main() {
    var wg sync.WaitGroup
    wg.Add(3)
    go func() {
        defer wg.Done()
        doThing1()
    }()
    go func() {
        defer wg.Done()
        doThing2()
    }()
    go func() {
        defer wg.Done()
        doThing3()
    }()
    wg.Wait()
}
```


Select on channels



select.go

```
select {  
case v := <-ch:  
    fmt.Println(v)  
case v := <-ch2:  
    fmt.Println(v)  
case ch3 <- x:  
    fmt.Println("wrote", x)  
case <-ch4:  
    fmt.Println("got value on ch4, but ignored it")  
}
```

codetoimg.com

You can go far with standard library

- **Networking and Web Development**
Build web servers and REST APIs using net/http.
- **Concurrency and Parallelism**
Use goroutines and channels for efficient concurrency.
- **File I/O and Filesystem Operations**
Handle file operations with os and io.
- **Data Encoding/Decoding (JSON, XML, etc.)**
Parse and generate JSON, XML, and CSV with encoding.
- **Cryptography and Security**
Utilize built-in cryptographic algorithms for secure communication.
- **Testing**
Write unit tests and benchmarks with the testing package.
- **Time and Date Handling**
Manage time zones, parsing, and duration with the time package.
- **Database Interaction**
Interact with SQL databases using database/sql.
- **Compression and Decompression**
Handle compressed files with gzip, zlib, and tar.
- **Text Processing**
Manipulate strings and use regular expressions.
- **Configuration Handling**
Use os for environment variables and flag for command-line flags.
- **Logging**
Perform basic logging with the log package.
- **Command Execution and Shell Interaction**
Run system commands using os/exec.

gopls, the Go language server

” Go Language Server is goooooood – autocomplete a couple of steps ahead



Rob Pike on Go

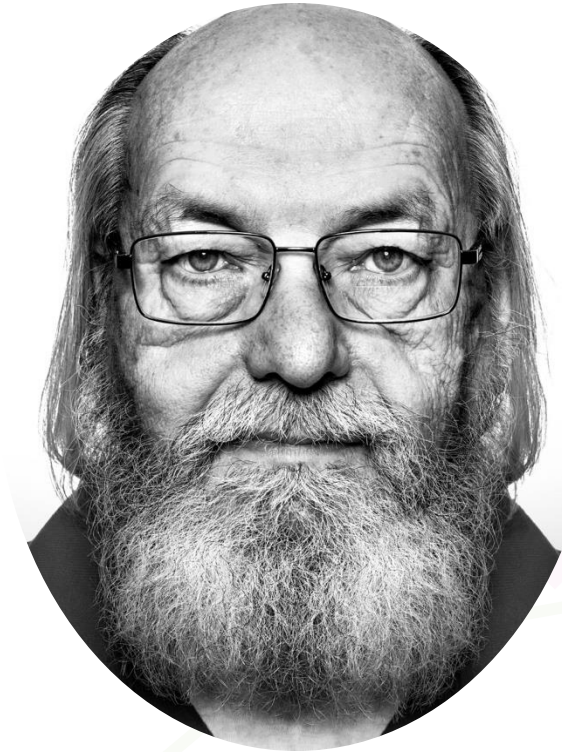


<https://youtu.be/yE5Tpp2BSGw>

- a strong **standard library** that implements most of the basics needed for server code
- **concurrency** as a first-class component of the language
- an approach based on **composition** rather than inheritance
- a **packaging model** that clarifies dependency management
- integrated fast build and **testing** tools
- rigorous consistent **formatting**
- a focus on **readability** over cleverness
- a **compatibility** guarantee

What Go did right – as seen by Rob Pike

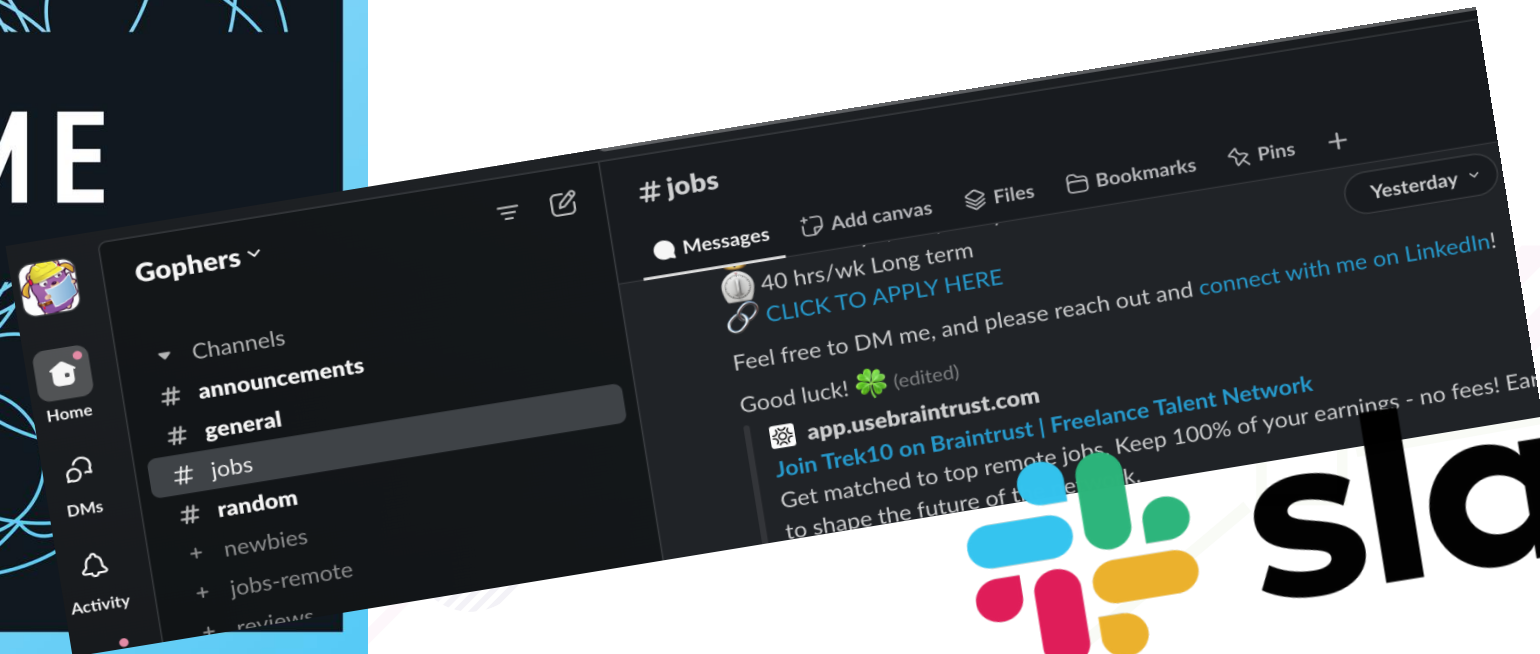
” One other crucial thing that Go got right, according to Pike, was its portability — that is, making it easy to compile code for other platforms. It was made possible in part by Ken Thompson’s compiler — written in the C programming language, even though others argued that the compiler should’ve been written in Go itself (or using the tools in LLVM).



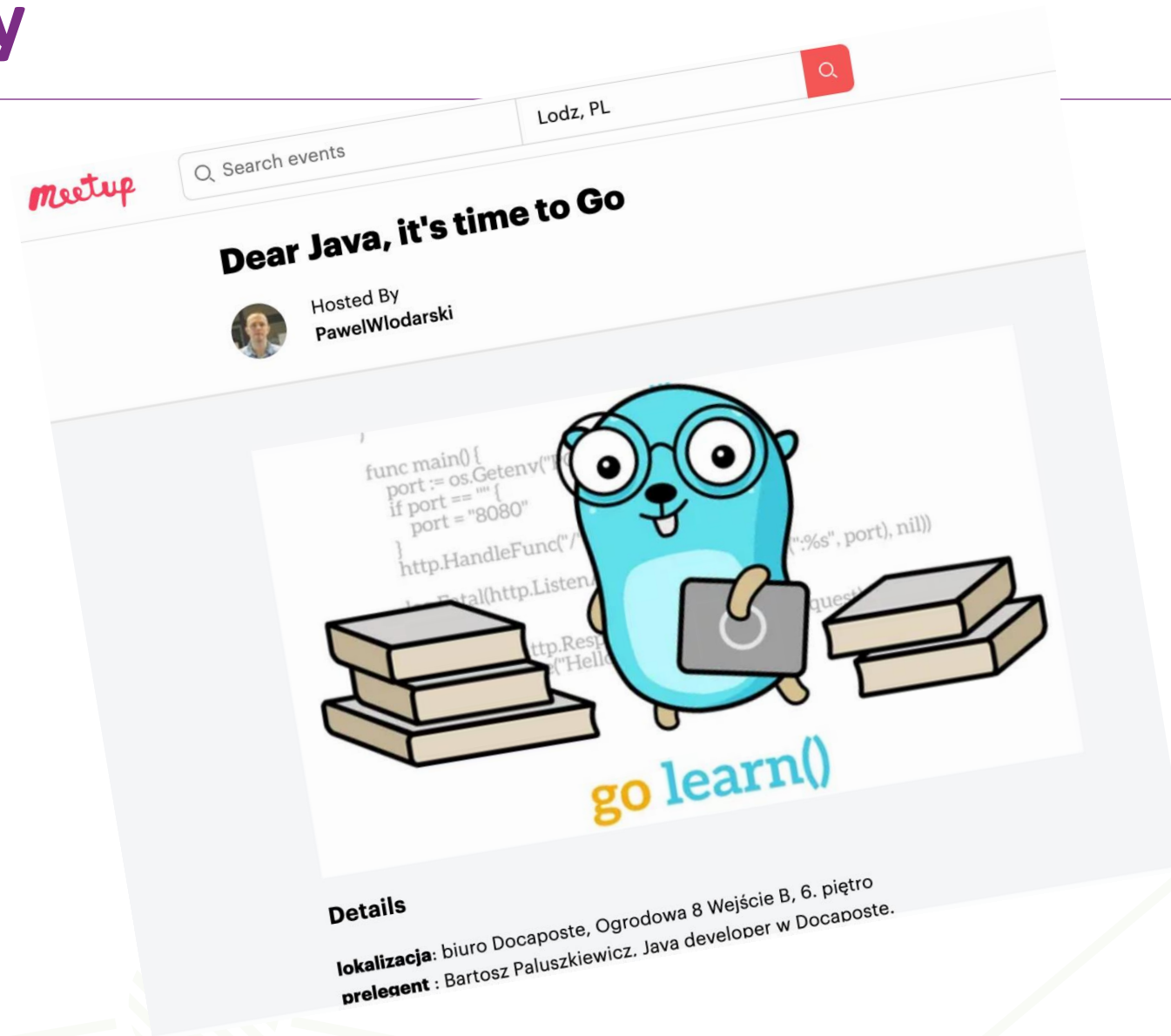
Community



GO TIME



Community



What's next aka „we haven't talked about”

- context
- mutex
- context cancellation pattern, select chan
- reflection
- more on unsafe
- cgo
- testing
- tooling
- immutability



Takeaways!

- Unlearn (don't be biased)
- Don't try to “re-create”
- Cherish new languages (even if not on prod)
- Seek for idiomatic way (don't fight the tool)
- Be inspired by new concepts
- You are not too old ;)



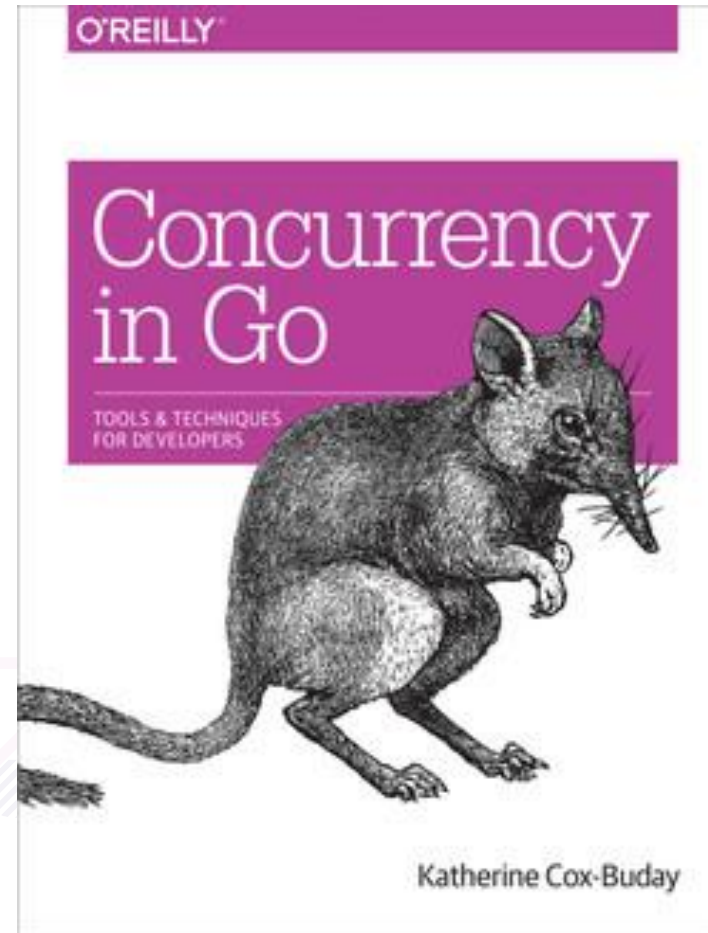
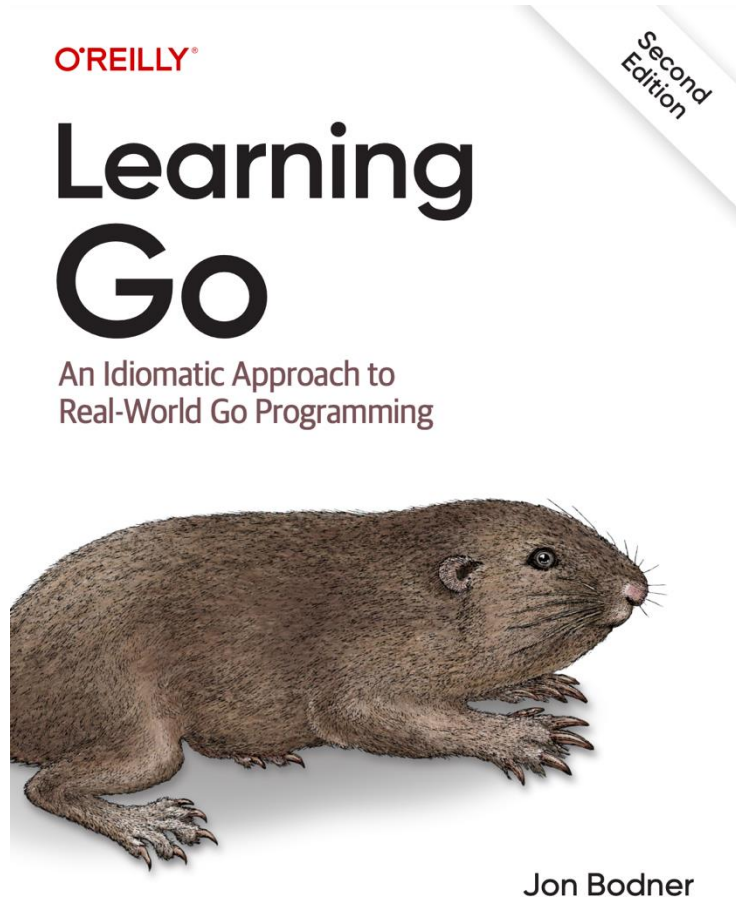


The background features a central horizontal band of purple and dark blue. Above this band, a green line with an arrow points downwards towards the center. To the left, a blue square with diagonal stripes is partially visible. To the right, a pink triangle and a blue circle with diagonal stripes are partially visible. Below the band, a blue semi-circle is on the left, and a blue triangle with a pink circle inside is on the right. At the bottom, a green line with a square at its end points upwards towards the center, and a green circle with diagonal stripes is on the left.

Notable resources

Golang goodies – Books

- <https://www.oreilly.com/library/view/learning-go-2nd/9781098139285/> - Learning Go, Jon Bodner
- <https://www.oreilly.com/library/view/concurrency-in-go/9781491941294/> - Concurrency in Go, Katherine Cox-Buday



Golang goodies – where to GO?

- <https://100go.co/> - 100 Go mistakes
- <https://go.dev/play/> - The Go Playground
- <https://go.dev/tour/welcome/1> - Tour of Go
- <https://fireship.io/lessons/learn-go-in-100-lines/> - learn Go in 100 lines by Fireship
- <https://www.coursera.org/specializations/google-golang> - Coursera specialization on Go
- <https://youtu.be/qYAHrud98Ek> - Go Go Java Developer by Piotr Przybył
- https://go.dev/doc/effective_go - Effective Go, how to write clean, idiomatic Go code
- <https://open.spotify.com/show/2cKdcxETn7jDp7uJCwqmSE?si=03b5720126b44205> – Go Time podcast
- <https://www.oreilly.com/library/view/learning-go-2nd/9781098139285/> - Learning Go, Jon Bodner
- <https://threedots.tech/> - courses, blogs and more



Golang goodies – beyond!





**That's all Folks!
Happy Coding!**

**_experience digital
transition!**